IUT Saint-Dié-des-Vosges

UNIVERSITÉ DE LORRAINE

Bases de la bibliothèque OpenCV
TD1

Hugo Rositi*

23 septembre 2026

Introduction

OpenCV (Open Source Computer Vision Library : http://opencv.org) est une bibliothèque multi-plateforme qui permet de développer des applications de vision par ordinateur en temps réel. Elle se concentre principalement sur le traitement d'images, la capture et l'analyse de vidéos, elle propose également des fonctions telles que la détection de visages et d'objets.

La vision par ordinateur peut être définie comme une discipline qui explique comment reconstruire et comprendre une scène en 3D à partir de ses images en 2D, en termes de propriétés de la structure présente dans la scène. Elle traite de la modélisation et de la reproduction de la vision humaine à l'aide de logiciels et de matériel informatique.

La vision par ordinateur recoupe de manière significative les domaines suivants :
- Traitement d'images : Il se concentre sur la manipulation d'images.
- Reconnaissance des formes : Elle explique les différentes techniques de classification des formes.
- Photogrammétrie : Il s'agit d'obtenir des mesures précises à partir d'images, des nuages de points 3D des surfaces des objets de la scène, par exemple.

1 Installation

Commencez par installer sur votre machine OpenCV-Python qui est l'API Python d'OpenCV. Assurez vous d'avoir un environnement de travail Python correctement configuré. N'oubliez pas de créer un environnement virtuel spécifique pour OpenCV.
Guide d'installation OpenCV-Python.

Afin de vérifier votre installation, exécuter le code suivant :

```
1 import cv2 as cv
2 print(cv.__version__)
3 print("OpenCV Version: {}".format(cv.__version__))
```

*hugo.rositi@univ-lorraine.fr

2 Fonctionnalités de base dans *OpenCV*

Dans cette séquence d'exercices simples vous allez apprendre comment charger, afficher et enregistrer des images et des vidéos, comment dessiner des formes simples, comment contrôler les événements de la souris et comment créer des trackbars.

2.1 Charger, afficher et enregistrer une image

Mettre en place une application qui vous permet de charger une image, de l'afficher et de l'enregistrer sous un autre nom en niveaux de gris ou en couleur.

2.2 Lire et visionner une vidéo

Programmez une application qui permet de lire une vidéo.

2.3 Lire et visionner le flux de la caméra

Programmez une application qui permet de lire et d'afficher le flux vidéo de la caméra.

2.4 Fonctions de traitements d'images

Étudiez les différentes fonctions de traitements d'images fournies par la bibliothèque.

1. Charger une image en couleurs.
2. Ajuster le niveau de quantification.
3. Ajoutez du bruit gaussien.
4. Calculer les contours présents à l'aide de l'opérateur de Canny.
5. Faire une dilatation morphologique de l'image des contours avec un masque de 5x5.
6. Faire une érosion de cette dernière avec le même masque.
7. Changer la taille de l'image avec la fonction `resize`
8. Découper l'image directement en tenant compte des propriétés matrice : `img[<debutHauteur:finHauteur>,<debutLargeur:finLargeur>]`

2.5 Dessiner des formes simples : cercle, ligne, rectangle, texte

Apprenez à dessiner différentes formes géométriques 2D fournies par la bibliothèque.

1. Commencer par créer une image noire de dimension $512x512$ avec 3 canaux pour les couleurs, de type *unsigned int* (`np.uint8`) en utilisant une matrice de zéros de la librairie `numpy`.
2. Colorier une partie de cette image en bleu en utilisant un intervalle.
3. Dessiner une ligne sur votre image en utilisant la fonction `cv.line`.
4. Dessiner un rectangle sur l'image avec la fonction `cv.rectangle`.
5. Dessiner un cercle plein avec la fonction `cv.circle`.
6. Ajouter un texte sur votre image avec la fonction `cv.putText`.
7. Voir dans la documentation de la bibliothèque quelles sont les autres formes géométriques simples et les expérimenter.

2.6 Réaliser un dessin

Dessiner une cible de tir à l'arc dans une image $256x256$ avec les fonctions vues précédemment.
Bonus : Quand vous cliquez sur l'image un texte de score se modifie.

3 Gestion des évéments

3.1 Gérer les événement de la souris

Dans cette section, vous allez apprendre comment gérer les événements de la souris dans *OpenCV*. Pour ce faire, commencez par afficher la liste des différents actions pouvant être prises en compte avec les instructions suivantes :

```
events = [i for i in dir(cv) if 'EVENT' in i]
print(events)
```

Maintenant, vous allez créer une application graphique qui permet de dessiner des cercles, en faisant un double-click, sur une image. Tout d'abord, vous allez créer une fonction de rappel pour la souris qui est exécutée lorsqu'un événement de souris se produit. Un événement de souris peut être n'importe quoi lié à la souris, comme le bouton gauche enfoncé, le bouton gauche relâché, un double-clic sur le bouton gauche, *etc.* . Cela nous donne les coordonnées (x, y) pour chaque événement de souris. Avec cet événement et cette position, nous pouvons faire ce que nous voulons.

Créer une fonction de rappel pour la souris a un format spécifique qui est le même partout. Elle ne diffère que par ce que la fonction accomplit. Dans notre cas, notre fonction de rappel pour la souris fait une seule chose : elle dessine un cercle là où nous double-cliquons. Voici donc le code ci-dessous :

```
# mouse callback function
def draw_circle(event,x,y,flags,param):
        if event == cv.EVENT_LBUTTONDBLCLK:
                cv.circle(img,(x,y),10,(255,0,0),-1)
# Create a black image, a window and
# bind the mouse callback function to this window
img = np.zeros((512,512,3), np.uint8)
cv.namedWindow('image')
cv.setMouseCallback('image',draw_circle)

while(1):
        cv.imshow('image',img)
        if cv.waitKey(1) & 0xFF == ord('q'):
                break
```

Pour aller plus loin , vous pouvez ajouter la possibilité de dessiner des cercles ou des rectangles, en fonction du mode choisi par l'utilisateur. Pour ce faire, il faut définir dans la boucle principale, un "binding" clavier pour la touche 'm' afin de basculer entre le rectangle et le cercle.

3.2 Créer des trackbars

Dans cette section, vous allez apprendre comment faire des *trackbars* ou barre de défilement dans *OpenCV*. Maintenant, vous allez créer une application simple qui montre la couleur que vous spécifiez avec des trackbars. Vous allez avoir une fenêtre qui affiche la couleur choisie et les trois barres de défilement pour spécifier chacune des couleurs R,G,B. En faisant glisser la barre de défilement, la couleur de la fenêtre change en conséquence. Par défaut, la couleur initiale sera définie sur noir.

Pour la fonction `cv.createTrackbar()`, le premier argument est le nom de la barre de défilement, le deuxième est le nom de la fenêtre à laquelle elle est attachée, le troisième argument est la valeur par défaut, le quatrième est la valeur maximale et le cinquième est la fonction de rappel qui est exécutée chaque fois que la valeur de la barre de défilement change.

La fonction de rappel (callback) a toujours un argument par défaut qui est la position de la barre de défilement. Dans notre cas, la fonction ne fait rien, donc vous allez passer simplement.

Information :
Une autre application importante de la barre de défilement est de l'utiliser comme un **bouton** ou un **interrupteur**. La librairie OpenCV, par défaut, n'a pas de fonctionnalité de bouton. On peut donc utiliser la barre de défilement pour obtenir une telle fonctionnalité. Dans cette application, vous allez créer un tel interrupteur pour que l'application fonctionne uniquement si l'interrupteur est ACTIVÉ, sinon l'écran est toujours noir.

3.3 Paint

Créez une application Paint avec des couleurs (RGB) et un rayon de pinceau réglables à l'aide de trackbars.